1-a)

Ok, eu vou explicar as 3 formas que encontrei para resolver isto. A pior de todas é comparar todos os pares possíveis, o que tem uma complexidade de merda (tipo task 4 do projeto) que é O(n^2).

A segunda maneira, que é a que eu usei para o 1-b, foi ordenar o array com um qsort e depois ir fazendo a diferença de um par com o outro até encontrar o par que eu quero. Percebe-se melhor no código o que isto é suposto fazer. Isto tudo mete complexidade O(N log N) no qsort e O(N) na cena dos pares.

A terceira maneira é usando uma hash, que nós ainda não aprendemos, mas vamos aprender. É basicamente uma tabela com duas colunas, uma que é o index e outra que é o actual valor nesse index. A lógica seria fazer array[i] + target e ver se existia na tabela para fazer os pares. A complexidade disto.... não sei bem honestamente, mas também acho que não precisamos de saber o que é por isso é chill

1-b)

```c
int comp(const void *a, const void *b){
    return (*(int *)a - *(int *)b);
}

void find_diff_tuples(int *arr, int N, int target, int** tuples, int *t_count){
    *t_count = 0;

    qsort(arr, N, sizeof(int), comp);

    *tuples = malloc(N*2 * sizeof(int));

    int left = 0;
    int right = 1;

    while(right<N){
        int diff = arr[right]-arr[left];

        if(diff == target){
            (*tuples)[*t_count*2] = arr[left];
            (*tuples)[*t_count*2+1] = arr[right];
            (*t_count)++;
            left++;
            right++;
        }
        else if(diff < target) right++;
        else{
            left++;
            if(left == right) right++;
        }

    }
    free(*tuples);
    if(*t_count == 0) *tuples = NULL;
}
```

1-c) não sei :))))

2-a)

2-1)

```c
void sort(Item arr[], int l, int r, int (*less) (Item, Item))
{
  int swapped = 1;
  int i, j = 0;
  Item tmp;

  while (swapped) {
    swapped = 0;
    for (i = r-1; i >= (l + j); i--) {

      /*==== TODO ====*/

      if (less(arr[i+1], arr[i])) {

        /*==== TODO ====*/

        tmp = arr[i];
        arr[i] = arr[i + 1];
        arr[i + 1] = tmp;

        swapped = 1;
      }
    }
    j++;
  }

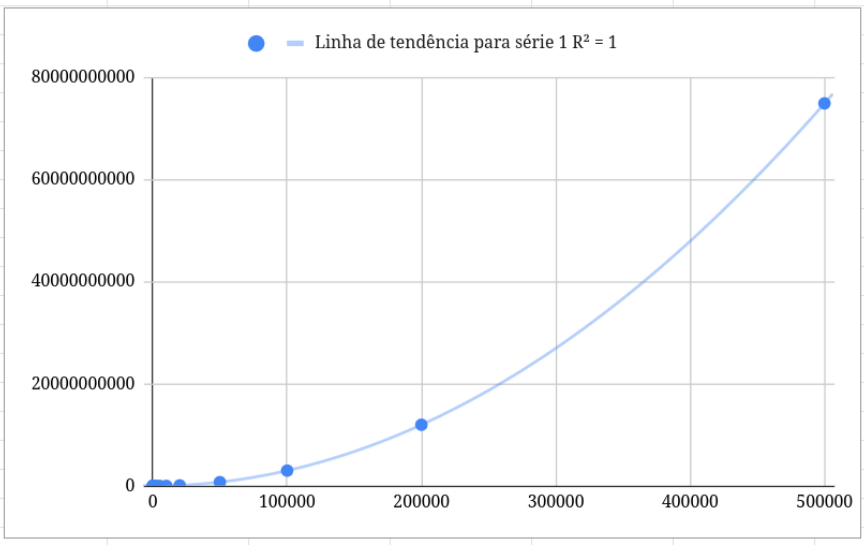
  return;
}
```

Este código incrivel apresenta o MELHOR e mais eficiente sorting de todos. O grande Bubble sort.
O código que eles pedem vou meter no githubi

2-2) codigo sigma aura github

2-4)

N	Acessos total
10	34
20	137
50	801
100	3233
200	12882
500	75886
1000	305480
2000	1217695
5000	7594723
10000	30748799
20000	119672177
50000	750396335
100000	2981045319
200000	11986833316
500000	74979142341

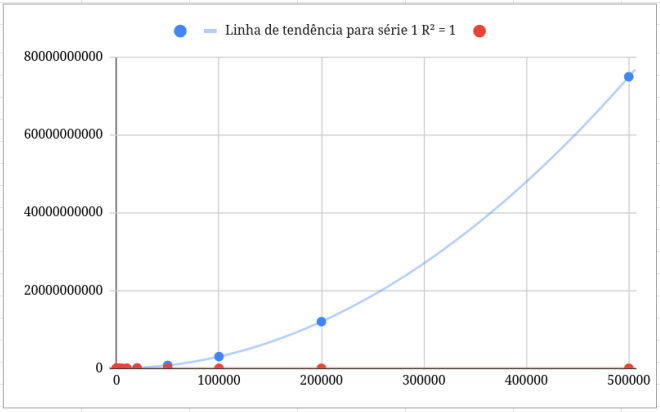


2-5) código no github

2-6)

Bubble Sort

N	Acessos total
10	34
20	137
50	801
100	3233
200	12882
500	75886
1000	305480
2000	1217695
5000	7594723
10000	30748799
20000	119672177
50000	750396335
100000	2981045319
200000	11986833316
500000	74979142341



Quick Sort

N	Acessos total
10	20
20	62
50	193
100	422
200	1302
500	2755
1000	7505
2000	15751
5000	45265
10000	95523
20000	208222
50000	604718
100000	1307632
200000	2800024
500000	7556689